

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_416cc0e5-a35\agent-a1ac8dcdf9e2793c0.jsonl`
- SHA-256: `83d0e165adf5dcd111c938fc7e52964fbdcbc31efc2f35e670b5dbd0e01c6e5f`
- Source modified: `2026-06-22T09:50:45+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_416cc0e5-a35`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:48:57.951Z)

Review the NIGHTLY REEL-COUNT engine-regression change under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e (a NEW, additive PR). It runs the configured WASB pipeline on a GCP GPU VM nightly, asserts reel-count (==) + quality metrics + cost, emails a report, self-deletes. Files: scripts/nightly_reelcount.py, scripts/nightly_email.py, deploy/gcp/nightly_regression/run_on_vm.sh, deploy/gcp/nightly_regression/launch.sh, deploy/gcp/nightly_regression/register_scheduler.sh, deploy/gcp/nightly_regression/Dockerfile.launcher, eval_baselines/reelcount_manifest.json. Read them under that absolute path. Cite file:line. Report only REAL defects (correctness/safety/security that would bite in production or a regression that escapes), not style. Return raw structured findings.

ADVERSARIALLY VERIFY this finding ? confirm with concrete code evidence under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e or REFUTE it (default real=false if the evidence isn't clearly there). Finding:

```
{
  "title": "CRITICAL: Failure return from segmenter is not a Success/Failure union, it's motion.py only (line 31 returns bare Failure, line 37 returns bare Success)",
  "file": "C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/motion.py:20-37",
  "severity": "high",
  "detail": "The motion.py's process_video() type hint declares 'List[Dict[str, Any]]' (line 20) but actually returns Success/Failure (lines 31, 37). This is a type contract violation. The docstring says 'Result[...] == Success | Failure' but the return type annotation is wrong. The nightly runner HANDLES this correctly at line 347-349 via isinstance(result, Failure) and .value fallback, but a caller expecting a bare list would crash with TypeError when calling len() on a Success object.",
  "fix": "Fix motion.py line 20: change return type hint from 'List[Dict[str, Any]]' to 'Result[List[Dict[str, Any]]]' (or 'Union[Success[List[Dict[str, Any]]], Failure, List[Dict[str, Any]]]' for full backward compat). The runtime behavior is correct; only the type hint is stale."
}
```

2. assistant (2026-06-22T09:49:01.298Z)

I'll review this regression change to verify the claimed type hint defect in motion.py. Let me start by examining the files in the nightly-e2e worktree.

3. user (2026-06-22T09:49:01.862Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-e2e'

4. assistant (2026-06-22T09:49:04.274Z)

Let me list the files in the worktree first:

5. user (2026-06-22T09:49:05.431Z)

```
total 303
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:10 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:05 ..
-rw-r--r-- 1 avidu 197609 86016 Jun 22 15:10 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 22 14:36 .coveragerc
-rw-r--r-- 1 avidu 197609    87 Jun 22 14:36 .git
-rw-r--r-- 1 avidu 197609   947 Jun 22 14:36 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .github
-rw-r--r-- 1 avidu 197609  1117 Jun 22 14:36 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:08 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:44 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:07 .ruff_cache
-rw-r--r-- 1 avidu 197609  5525 Jun 22 14:36 CLAUDE.md
-rw-r--r-- 1 avidu 197609 35347 Jun 22 14:36 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:43 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:43 backend
-rw-r--r-- 1 avidu 197609  2735 Jun 22 14:36 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 deploy
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:07 docs
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:05 eval_baselines
-rw-r--r-- 1 avidu 197609  2344 Jun 22 14:36 mypy.ini
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:10 output
-rw-r--r-- 1 avidu 197609  4525 Jun 22 14:36 pyproject.toml
-rw-r--r-- 1 avidu 197609   245 Jun 22 14:36 pytest.ini
-rw-r--r-- 1 avidu 197609   642 Jun 22 14:36 requirements.txt
-rw-r--r-- 1 avidu 197609   884 Jun 22 14:36 ruff.toml
-rwxr-xr-x 1 avidu 197609  2055 Jun 22 14:36 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5620 Jun 22 14:36 run_eval.py
-rw-r--r-- 1 avidu 197609  4356 Jun 22 14:36 run_regression.py
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 scratch
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:11 scripts
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:11 tests
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:43 tools
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:44 training
```

6. assistant (2026-06-22T09:49:07.618Z)

Now let me read the motion.py file to verify the claimed type hint defect:

7. user (2026-06-22T09:49:08.786Z)

```
1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
```

```

20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)
29             if not cap.isOpened():
30                 logger.error(f"Segmenter could not open video: {video_path}")
31                 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33             final_segments = self._detect_motion_segments(cap, max_frames)
34
35             segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37             return Success(value=segments_data)
38
39         def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40             """Run the pixel-motion heuristic over ``cap`` and return the detected
41             (start, end) play segments.
42
43             Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44             thresholds it into raw segments, then merges dead-time gaps and filters
45             out segments shorter than ``min_segment_duration``. ``cap`` is released
46             before returning. This is the detection half of ``process_video`` ? it
47             performs no DB writes and no fabrication.
48             """
49             fps = cap.get(cv2.CAP_PROP_FPS)
50             total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51             duration = total_frames / fps if fps > 0 else 0
52
53             # Sub-sample settings (5 FPS for analysis)
54             analysis_fps = 5.0
55             frame_interval = max(1, int(fps / analysis_fps))
56
57             idx_cfg = self.config.get("indexing", {})
58             motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59             min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60             dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
61
62             prev_gray = None
63             motion_signals = []
64             timestamps = []
65
66             frame_idx = 0
67             processed_count = 0
68             t_start = time.time()
69             # Emit a progress line roughly every 5% of the video so long runs are
70             # observable instead of silent (a full 1080p match is ~tens of minutes).
71             progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
72             next_progress = progress_step
73             logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
74                 f"({~{duration/60:.1f} min), analyzing every {frame_interval}th
frame")
75             while True:
76                 # Skip decoding for intermediate frames using cap.grab() for high
performance
77                 for _ in range(frame_interval - 1):
78                     if not cap.grab():

```

```

79         break
80         frame_idx += 1
81
82     ret, frame = cap.read()
83     if not ret:
84         break
85
86     # Process frame
87     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
88     # Apply Gaussian Blur to smooth noise
89     gray = cv2.GaussianBlur(gray, (21, 21), 0)
90
91     t = frame_idx / fps
92     timestamps.append(t)
93
94     if prev_gray is not None:
95         # Frame absolute difference (highly optimized motion tracking)
96         diff = cv2.absdiff(prev_gray, gray)
97         _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
98
99         # Dilate to fill gaps
100        thresh = cv2.dilate(thresh, None, iterations=2)
101
102        # Motion ratio
103        motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
104        motion_signals.append(motion_ratio)
105    else:
106        motion_signals.append(0.0)
107
108    prev_gray = gray
109    frame_idx += 1
110    processed_count += 1
111
112    if progress_step and frame_idx >= next_progress:
113        pct = 100.0 * frame_idx / total_frames
114        elapsed = time.time() - t_start
115        eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
116        logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
117                  f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
118        next_progress += progress_step
119
120    if max_frames is not None and processed_count >= max_frames:
121        logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
122        break
123
124    if frame_idx >= total_frames:
125        break
126
127    cap.release()
128
129    # Smooth signal using a moving average
130    window_size = 5
131    smoothed = []
132    for i in range(len(motion_signals)):
133        start = max(0, i - window_size // 2)
134        end = min(len(motion_signals), i + window_size // 2 + 1)
135        smoothed.append(sum(motion_signals[start:end]) / (end - start))
136
137    # Detect active play boundaries
138    in_segment = False
139    segment_start = None

```

```

140     raw_segments: List[Tuple[float, float]] = []
141
142     for i, t in enumerate(timestamps):
143         signal = smoothed[i]
144         if signal > motion_threshold:
145             if not in_segment:
146                 in_segment = True
147                 segment_start = t
148             else:
149                 if in_segment:
150                     in_segment = False
151                     raw_segments.append((segment_start, t))
152
153     # Handle video ending while in active segment
154     if in_segment:
155         raw_segments.append((segment_start, timestamps[-1]))
156
157     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
158     merged_segments: List[Tuple[float, float]] = []
159     if raw_segments:
160         curr_start, curr_end = raw_segments[0]
161         for next_start, next_end in raw_segments[1:]:
162             if next_start - curr_end < dead_time_merge_gap:
163                 # Merge
164                 curr_end = next_end
165             else:
166                 merged_segments.append((curr_start, curr_end))
167                 curr_start, curr_end = next_start, next_end
168         merged_segments.append((curr_start, curr_end))
169
170     # Post-Processing: 2. Filter segments shorter than min_segment_duration
171     final_segments: List[Tuple[float, float]] = [
172         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
173     ]
174
175     return final_segments
176
177 def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
178                             player_pool: List[str]) -> List[Dict[str, Any]]:
179     """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181     For each detected segment this assigns a random server/receiver, fabricates
182     rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183     with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184     Returns the per-segment dicts handed back to the caller. The ``random``
185     call order here is load-bearing for reproducibility and must not change.
186     """
187     # Populate SQLite DB with segments and metadata events
188     segments_data = []
189     for start, end in final_segments:
190         # Assign Server and Receiver randomly from pool
191         server, receiver = random.sample(player_pool, 2)
192
193         # Formulate dynamic rich tags
194         dur = end - start
195         metadata = {
196             "shot_count": random.randint(4, 25),
197             "intensity": "high" if dur > 12 else "medium",
198             "avg_speed_kmh": round(random.uniform(40, 85), 1)
199         }
200
201         segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202

```

```

203         if segment_id:
204             # Add sub-segment events
205             # A: Serve Event (always at start)
206             serve_foul = random.random() < 0.15 # 15% chance of serve foul
207             self.db.add_event(
208                 segment_id=segment_id,
209                 timestamp=start + 0.5,
210                 event_type="serve",
211                 player=server,
212                 details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213             )
214
215             # B: Smash Events (1 to 3 random smashes per segment)
216             smashes_count = random.randint(0, 3)
217             for _ in range(smashes_count):
218                 smash_time = random.uniform(start + 1.5, end - 1.0)
219                 smasher = random.choice([server, receiver])
220                 self.db.add_event(
221                     segment_id=segment_id,
222                     timestamp=round(smash_time, 2),
223                     event_type="smash",
224                     player=smasher,
225                     details={"speed_kmh": round(random.uniform(120, 250), 1)}
226                 )
227
228             # C: Segment Termination Event (foul/winner)
229             foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230             is_foul = random.random() > 0.4
231             error_player = random.choice([server, receiver])
232
233             self.db.add_event(
234                 segment_id=segment_id,
235                 timestamp=end - 0.2,
236                 event_type="foul" if is_foul else "winner",
237                 player=error_player,
238                 details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
239             )
240
241             # Store segment representation to return to caller
242             segments_data.append({
243                 "id": segment_id,
244                 "start_time": start,
245                 "end_time": end,
246                 "duration": dur,
247                 "server": server,
248                 "receiver": receiver,
249                 "metadata": metadata
250             })
251
252         return segments_data
253
254     # Register the segmenter
255     segmenter_registry.register(MotionPlaySegmenter)
256

```

8. assistant (2026-06-22T09:49:11.491Z)

Great! I can see the defect clearly at line 20. Let me now check how the nightly runner handles this and also examine the Success/Failure result types:

9. user (2026-06-22T09:49:12.443Z)

```

1         """Lightweight Result type for standardizing success vs. failure across the pipeline.

```

```

2
3     This is the start of the Error/Result Contract standardization (low-regret item #3).
4
5     The goal is to make it impossible to confuse "no data / no rallies" with "something
actually failed"
6     (e.g. missing API key, WSL healthcheck failure, provider error, etc.).
7
8     We use simple dataclasses so callers can use isinstance checks or structural pattern
matching.
9     This is intentionally small and incremental ? we start by applying it to the segmenter
10    and provider layers (the biggest sources of silent [] returns today).
11
12    The types are generic where it makes sense. Callers can still access .value on Success
13    or .message on Failure.
14
15    This module has no dependencies on the rest of the app.
16    """
17
18    from __future__ import annotations
19
20    from dataclasses import dataclass, field
21    from typing import Generic, TypeVar, List, Optional, Any, Tuple, Union
22
23    T = TypeVar("T")
24
25
26    @dataclass
27    class Success(Generic[T]):
28        """Represents a successful outcome with an optional list of non-fatal warnings."""
29
30        value: T
31        warnings: List[str] = field(default_factory=list)
32
33        def is_success(self) -> bool:
34            return True
35
36        def unwrap(self) -> T:
37            """Return the value. Only safe to call if you already know it's a Success."""
38            return self.value
39
40
41    @dataclass
42    class Failure:
43        """Represents a failure with a human-readable message and optional details."""
44
45        message: str
46        details: Optional[dict[str, Any]] = None
47        is_recoverable: bool = False # e.g. missing API key (recoverable) vs internal crash
48
49        def is_success(self) -> bool:
50            return False
51
52        def unwrap(self) -> Any:
53            """Raise to make misuse obvious in code that assumes success."""
54            raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
55
56
57    # Convenient alias for use in type hints: Result[list[dict], Failure]
58    # Result[ValueType] means Success[ValueType] | Failure
59    # (the Failure "error" type is fixed for this contract)
60    Result = Union[Success[T], Failure]
61
62
63    def unwrap_or_failure(result: Any) -> Tuple[Optional[Any], Optional[Failure]]:
64        """Split a segmenter/provider return into ``(value, failure)`` so a caller branches

```

```

once.
65
66     - ``Failure``          -> ``(None, failure)``
67     - ``Success(value=v)`` -> ``(v, None)``
68     - a bare value (legacy code that predates this contract and still returns a raw
list)
69         -> ``(value, None)``
70
71     This is the same isinstance/``.value`` unwrap that ``backend/main.py`` does inline;
callers
72     that forget it (e.g. ``len(process_video(...))``) crash with ``TypeError: ... has no
len()``
73     on a ``Success`` dataclass. ``result`` is typed ``Any`` because the contract is
heterogeneous
74     (Success | Failure | legacy bare value)."""
75     if isinstance(result, Failure):
76         return None, result
77     value = result.value if hasattr(result, "value") else result
78     return value, None
79

```

10. user (2026-06-22T09:49:13.727Z)

```

1     """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3     Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves
4     it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5     vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6     trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7     needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8     owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10    What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11    ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12    Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13    **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14    ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15    ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16    ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18    Extensible pattern (add a video = one manifest row + re-freeze):
19    ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20    ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22    Usage
23    -----
24    Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25    python scripts/nightly_reelcount.py --update-baseline

```



```

26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28         python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33     from __future__ import annotations
34
35     import argparse
36     import datetime as _dt
37     import json
38     import os
39     import sys
40     from dataclasses import dataclass, field
41     from typing import Any, Dict, List, Optional, Tuple
42
43     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
44     if REPO_ROOT not in sys.path:
45         sys.path.insert(0, REPO_ROOT)
46
47     # billing is pure (typing only) ? safe to import at module top.
48     from backend import billing # noqa: E402
49
50     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
51     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
52     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
53
54     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
tolerance.
55     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
56     DEFAULT_QUALITY_TOL = 0.02 # quality is float/labeled ? a looser tripwire than the
exact reel-count
57     DEFAULT_FX_INR_PER_USD = 83.0
58     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
south1, ~mid-2026).
59     DEFAULT_MACHINE_USD_PER_HR = 0.35
60
61
62     @dataclass(frozen=True)
63     class Tolerances:
64         quality_tol: float = DEFAULT_QUALITY_TOL
65
66         def to_dict(self) -> Dict[str, Any]:
67             return {"quality_tol": self.quality_tol}
68
69
70     # ----- #
71     # Pure cost math (wraps backend.billing ? no IO).
72     # ----- #
73     def cost_report(billed_seconds: float, machine_usd_per_hr: float,
74                    fx_inr_per_usd: float, gpu_seconds: float = 0.0) -> Dict[str, Any]:
75         """Run cost from billed wall-seconds × machine rate ? USD + INR (`backend.billing`
arithmetic).
76
77         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
``--vm-uptime-seconds``;
78         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
for boot/install)."""
79         rate_per_sec = float(machine_usd_per_hr) / 3600.0
80         usage = {billing.WALL_SECONDS: float(billed_seconds)}
81         rates = {billing.WALL_SECONDS: rate_per_sec}

```

```

82     usd, breakdown = billing.compute_cost_usd(usage, rates)
83     return {
84         "billed_seconds": round(float(billed_seconds), 1),
85         "gpu_seconds": round(float(gpu_seconds), 1),
86         "machine_usd_per_hr": float(machine_usd_per_hr),
87         "usd": usd,
88         "inr": billing.to_inr(usd, fx_inr_per_usd),
89         "breakdown": breakdown,
90     }
91
92
93     # ----- #
94     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
95     # ----- #
96     def summarize_results(run_results: List[Dict[str, Any]], segmenter: str,
97                           cost: Dict[str, Any]) -> Dict[str, Any]:
98         """Compact, comparable snapshot from per-video run results.
99
100         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
101 wall_seconds}``.
102         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
103 never hides it)."""
104         per_video: Dict[str, Any] = {}
105         for r in run_results:
106             per_video[r["name"]] = {
107                 "reel_count": r.get("reel_count"),
108                 "ok": bool(r.get("ok", False)),
109                 "error": r.get("error"),
110                 "quality": r.get("quality"),
111             }
112         return {"segmenter": segmenter, "cost": cost, "per_video": per_video,
113               "n": len(per_video)}
114
115
116 @dataclass
117 class Finding:
118     video: str
119     metric: str          # "reel_count" | "error" | a quality key | "segmenter" |
120     "corpus"
121     kind: str            # "regression" | "improvement" | "stale"
122     baseline: Optional[float] = None
123     current: Optional[float] = None
124     detail: str = ""
125
126     def message(self) -> str:
127         if self.kind == "stale":
128             return f"[STALE] {self.metric}: {self.detail}"
129         if self.metric == "error":
130             return f"[REGRESSION] {self.video}: pipeline ERROR ? {self.detail}"
131         b = "?" if self.baseline is None else f"{self.baseline:g}"
132         c = "?" if self.current is None else f"{self.current:g}"
133         tag = "REGRESSION" if self.kind == "regression" else "improvement"
134         return f"[{tag}] {self.video}/{self.metric}: {b} ? {c}"
135
136
137 @dataclass
138 class NightlyResult:
139     findings: List[Finding] = field(default_factory=list)
140     added: List[str] = field(default_factory=list)
141     removed: List[str] = field(default_factory=list)
142
143     @property
144     def regressions(self) -> List[Finding]:
145         return [f for f in self.findings if f.kind == "regression"]

```

```

144     @property
145     def improvements(self) -> List[Finding]:
146         return [f for f in self.findings if f.kind == "improvement"]
147
148     @property
149     def stale(self) -> List[Finding]:
150         return [f for f in self.findings if f.kind == "stale"]
151
152     def overall_status(self) -> str:
153         if self.regressions:
154             return "REGRESSION"
155         if self.stale:
156             return "STALE"
157         return "PASS"
158
159     def exit_code(self) -> int:
160         if self.regressions:
161             return 1
162         if self.stale:
163             return 4
164         return 0
165
166
167     def compare(baseline: Dict[str, Any], current: Dict[str, Any], tols: Tolerances) ->
NightlyResult:
168         """Diff current run vs frozen baseline.
169
170         * Segmenter changed ? STALE (numbers not comparable; re-freeze).
171         * Corpus (video set) changed ? STALE + report added/removed; still diff the
INTERSECTION.
172         * Per shared video: reel_count EXACT (any change = regression); a pipeline ERROR =
regression;
173         quality metrics drop beyond ``quality_tol`` = regression (rise = improvement).
174         """
175         res = NightlyResult()
176         if baseline.get("segmenter") != current.get("segmenter"):
177             res.findings.append(Finding("", "segmenter", "stale",
178                                     detail=f"{baseline.get('segmenter')} ?
{current.get('segmenter')}"))
179             return res
180
181         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
182         res.added = sorted(set(c_pv) - set(b_pv))
183         res.removed = sorted(set(b_pv) - set(c_pv))
184         if res.added or res.removed:
185             res.findings.append(Finding("", "corpus", "stale",
186                                     detail=f"added={res.added} removed={res.removed}"))
187
188         for v in sorted(set(b_pv) & set(c_pv)):
189             b, c = b_pv[v], c_pv[v]
190             # 1) pipeline must still succeed
191             if not c.get("ok", False) or c.get("reel_count") is None:
192                 res.findings.append(Finding(v, "error", "regression",
193                                     detail=str(c.get("error") or "no reel_count
produced")))
194                 continue
195             # 2) reel count ? EXACT
196             bc, cc = b.get("reel_count"), c.get("reel_count")
197             if bc is not None and cc is not None and cc != bc:
198                 res.findings.append(Finding(v, "reel_count", "regression", float(bc),
float(cc)))
199             # 3) quality metrics ? tolerance (only if both sides have them)
200             bq, cq = b.get("quality") or {}, c.get("quality") or {}
201             for m in QUALITY_HIGHER:
202                 bv, cv = bq.get(m), cq.get(m)

```

```

203         if bv is None or cv is None:
204             continue
205         if cv < bv - tols.quality_tol:
206             res.findings.append(Finding(v, m, "regression", bv, cv))
207         elif cv > bv + tols.quality_tol:
208             res.findings.append(Finding(v, m, "improvement", bv, cv))
209     return res
210
211
212     # ----- #
213     # Report formatting (pure).
214     # ----- #
215     def _q(qd: Optional[Dict[str, Any]], key: str) -> str:
216         if not qd or qd.get(key) is None:
217             return "?"
218         return f"{qd[key]:.3f}"
219
220
221     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
222                      result: NightlyResult, date: str, head: str) -> str:
223         status = result.overall_status()
224         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION",
225                 "STALE": "? STALE (re-freeze the baseline)"}[status]
226         cost = current.get("cost", {})
227         L: List[str] = [
228             f"# Nightly engine regression (reel-count + quality + cost) ? {date}",
229             "",
230             f"***Status: {badge}** · exit code {result.exit_code()} · segmenter
`{current.get('segmenter')}`",
231             "",
232             f"- HEAD `{head}` · baseline `{baseline.get('git_commit', '?')}` (frozen
{baseline.get('generated_at', '?')})",
233             f"- **Run cost: ${cost.get('usd', 0):.4f} (?{cost.get('inr', 0):.2f})** · "
234             f"billed {cost.get('billed_seconds', 0):.0f}s @ ${cost.get('machine_usd_per_hr',
0)}/hr",
235             "",
236         ]
237         if result.regressions:
238             L += ["## ? Regressions", ""] + [f"- {f.message()}" for f in result.regressions]
+ [""]
239         if result.stale:
240             L += ["## ? Stale / not comparable", ""] + [f"- {f.message()}" for f in
result.stale]
241             L += ["- Action: `python scripts/nightly_reelcount.py --update-baseline` (after
confirming the change is intended).", ""]
242
243         # Per-video table.
244         b_pv, c_pv = baseline.get("per_video", {}), current.get("per_video", {})
245         L += ["## Per-video", ""],
246             "| video | reels (base?now) | tol_f1 (base?now) | f1@0.5 | status |",
247             "|---|---|---|---|---|"]
248         for v in sorted(set(b_pv) | set(c_pv)):
249             b, c = b_pv.get(v, {}), c_pv.get(v, {})
250             bc = b.get("reel_count")
251             cc = c.get("reel_count")
252             rc = f"'?' if bc is None else bc?{'ERROR' if not c.get('ok', True) else ('?'
if cc is None else cc)}"
253             bq, cq = b.get("quality"), c.get("quality")
254             tf = f"_q(bq, 'tol_f1')?_q(cq, 'tol_f1')]"
255             f5 = f"_q(bq, 'f1@0.5')?_q(cq, 'f1@0.5')]"
256             vstatus = "?" if any(f.video == v and f.kind == "regression" for f in
result.findings) else "?"
257             L.append(f"| {v} | {rc} | {tf} | {f5} | {vstatus} |")
258         L.append("")
259

```

```

260         if result.improvements:
261             L += ["_Improvements over baseline (re-freeze to ratchet up):_" ] \
262                 + [f"- {f.message()}" for f in result.improvements] + [""]
263         L += ["---",
264             "_Full configured+checkpointed pipeline on the real model (GCP GPU VM).
Tripwire only ? does "
265             "not change the promotion gate. Complements PR #202's cached-trajectory CPU
re-score._"]
266         return "\n".join(L)
267
268
269     # ----- #
270     # IO shell ? running the configured pipeline per video (needs backend + GPU + video).
271     # ----- #
272     def _load_gts(labels_ref: Optional[str]) -> Optional[List[Tuple[float, float]]]:
273         """Load ground-truth rally intervals from a golden-features JSON (`gts` key) or a
rallies CSV
274         (`start,end` columns / pairs). Returns None when no labels are configured."""
275         if not labels_ref:
276             return None
277         from backend.storage import fetch
278         local = fetch(labels_ref)
279         if local.endswith(".json"):
280             with open(local, encoding="utf-8") as fh:
281                 data = json.load(fh)
282                 return [(float(s), float(e)) for s, e in (data.get("gts") or [])]
283         # CSV: header start,end (or first two numeric columns)
284         import csv
285         out: List[Tuple[float, float]] = []
286         with open(local, encoding="utf-8") as fh:
287             for row in csv.reader(fh):
288                 try:
289                     out.append((float(row[0]), float(row[1])))
290                 except (ValueError, IndexError):
291                     continue
292         return out
293
294
295     def _segments_to_intervals(segs: List[Dict[str, Any]]) -> List[Tuple[float, float]]:
296         """`start, end` pairs from segmenter result dicts, tolerant of the two key
conventions in the
297         codebase (`start_time`/`end_time` ? motion/DB shape ? or `start`/`end`).
Used only for the
298         optional quality metrics; the reel COUNT is `len(segs)` regardless, so a key
mismatch degrades
299         quality scoring gracefully (skipped) without ever miscounting reels."""
300         out: List[Tuple[float, float]] = []
301         for s in segs:
302             start = s.get("start_time", s.get("start"))
303             end = s.get("end_time", s.get("end"))
304             if start is not None and end is not None:
305                 out.append((float(start), float(end)))
306         return out
307
308
309     def run_one_video(video: Dict[str, Any], segmenter: str, config: Any,
310                      rerun_on_mismatch: bool = True,
311                      baseline_count: Optional[int] = None) -> Dict[str, Any]:
312         """Run the configured pipeline on one video ? reel count (+ quality vs labels). Lazy
backend imports.
313
314         The re-run-once guard (the one known non-determinism ? a transient
`add_play_segment`?None drop,
315         database.py): if the count != the frozen baseline, re-process the video ONCE; a
transient drop won't

```

```

316 reproduce, a real change will."""
317 import random
318 import time
319
320 from backend.eval import rally_seg_eval
321 from backend.infrastructure.database import Database
322 from backend.pipeline.segmenters import segmenter_registry
323 from backend.results import Failure
324 from backend.storage import fetch
325 from backend.utils.hashing import compute_video_id
326
327 name = video["name"]
328 try:
329     local = fetch(video["uri"])
330 except Exception as e: # noqa: BLE001 ? surface, never hide
331     return {"name": name, "reel_count": None, "ok": False, "error": f"fetch failed:
332 {e}",
333           "quality": None, "wall_seconds": 0.0}
334
335 gts = _load_gts(video.get("labels_uri"))
336 seg_cls = segmenter_registry.get(segmenter)
337 if not seg_cls:
338     return {"name": name, "reel_count": None, "ok": False,
339           "error": f"segmenter '{segmenter}' not registered", "quality": None,
340 "wall_seconds": 0.0}
341
342 def _process(db_path: str) -> Tuple[Optional[int], Optional[List[Tuple[float,
343 float]]], float, Optional[str]]:
344     random.seed(0) # close motion.py's unseeded metadata RNG so secondary fields
345 are reproducible too
346     db = Database(db_path)
347     video_id = compute_video_id(local)
348     t0 = time.monotonic()
349     result = seg_cls(db, config).process_video(video_id, local)
350     wall = time.monotonic() - t0
351     if isinstance(result, Failure):
352         return None, None, wall, getattr(result, "message", "segmenter failed")
353     segs = result.value if hasattr(result, "value") else result
354     intervals = _segments_to_intervals(segs)
355     return len(segs), intervals, wall, None
356
357 import tempfile
358 total_wall = 0.0
359 with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
360     count, intervals, wall, err = _process(os.path.join(td, "run1.db"))
361     total_wall += wall
362     if err is not None:
363         return {"name": name, "reel_count": None, "ok": False, "error": err,
364               "quality": None, "wall_seconds": round(total_wall, 2)}
365     # re-run-once guard for the transient DB-drop flake
366     if rerun_on_mismatch and baseline_count is not None and count != baseline_count:
367         count2, intervals2, wall2, err2 = _process(os.path.join(td, "run2.db"))
368         total_wall += wall2
369         if err2 is None and count2 is not None:
370             count, intervals = count2, intervals2 # the reproduced (or corrected)
371 value
372
373     quality = None
374     if gts and intervals is not None:
375         row = rally_seg_eval.score_intervals(intervals, gts)
376         quality = {k: row[k] for k in ("tol_f1", "f1@0.5", "f1@0.25",
377 "tol_precision", "tol_recall")}
378         if k in row}
379
380 return {"name": name, "reel_count": count, "ok": True, "error": None,

```

```

375         "quality": quality, "wall_seconds": round(total_wall, 2)}
376
377
378     def load_manifest(path: str) -> Dict[str, Any]:
379         with open(path, encoding="utf-8") as fh:
380             return json.load(fh)
381
382
383     def run_corpus(manifest: Dict[str, Any], baseline: Optional[Dict[str, Any]],
384                   rerun_on_mismatch: bool = True) -> Tuple[List[Dict[str, Any]], float]:
385         """Run every manifest video through the configured pipeline. Returns (run_results,
386         total_wall_s)."""
387         from backend.config import load_config
388
389         config = load_config()
390         # apply manifest config overrides onto the typed config (dotted keys)
391         for dotted, val in (manifest.get("config_overrides") or {}).items():
392             _set_dotted(config, dotted, val)
393         segmenter = manifest.get("segmenter") or getattr(getattr(config, "indexing", None),
394         "default_segmenter", "motion")
395         b_pv = (baseline or {}).get("per_video", {})
396         results: List[Dict[str, Any]] = []
397         total_wall = 0.0
398         for v in manifest.get("videos", []):
399             base_count = (b_pv.get(v["name"]) or {}).get("reel_count")
400             r = run_one_video(v, segmenter, config, rerun_on_mismatch, base_count)
401             total_wall += float(r.get("wall_seconds") or 0.0)
402             results.append(r)
403         return results, total_wall
404
405     def _set_dotted(config: Any, dotted: str, value: Any) -> None:
406         """Best-effort dotted-path set onto a pydantic config (e.g.
407         'indexing.motion_threshold')."""
408         parts = dotted.split(".")
409         obj = config
410         for p in parts[:-1]:
411             obj = getattr(obj, p, None)
412             if obj is None:
413                 return
414         try:
415             setattr(obj, parts[-1], value)
416         except Exception: # noqa: BLE001 ? overrides are best-effort; a bad key shouldn't
417             crash the run
418             pass
419
420     def _git_head() -> str:
421         import subprocess
422         try:
423             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
424             capture_output=True, text=True, timeout=10)
425             return out.stdout.strip() or "?"
426         except Exception: # noqa: BLE001
427             return "?"
428
429     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
430         if not os.path.exists(path):
431             return None
432         with open(path, encoding="utf-8") as fh:
433             return json.load(fh)
434
435     def save_baseline(path: str, snapshot: Dict[str, Any]) -> None:

```

```

436         out = dict(snapshot)
437         out["generated_at"] = _dt.date.today().isoformat()
438         out["git_commit"] = _git_head()
439         out["_doc"] = ("Frozen nightly reel-count baseline. Regenerate with `python
scripts/nightly_reelcount.py "
440                        "--update-baseline` after an INTENDED engine change or when adding a
video. Tied to the "
441                        "configured+checkpointed engine + the manifest corpus.")
442         out.pop("cost", None) # cost is per-run, not part of the frozen baseline
443         os.makedirs(os.path.dirname(path), exist_ok=True)
444         tmp = path + ".tmp"
445         with open(tmp, "w", encoding="utf-8") as fh:
446             json.dump(out, fh, indent=2, sort_keys=True)
447             fh.write("\n")
448         os.replace(tmp, path)
449
450
451     def build_parser() -> argparse.ArgumentParser:
452         p = argparse.ArgumentParser(description="Nightly E2E engine regression: reel-count +
quality + cost.")
453         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
454         p.add_argument("--baseline", default=DEFAULT_BASELINE)
455         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR)
456         p.add_argument("--quality-tol", type=float, default=DEFAULT_QUALITY_TOL)
457         p.add_argument("--vm-uptime-seconds", type=float, default=None,
458                        help="authoritative VM uptime (boot?delete) for the cost line; "
459                        "defaults to the in-process pipeline wall time (a lower bound)")
460         p.add_argument("--no-rerun-guard", action="store_true",
461                        help="disable the re-run-once-on-mismatch guard (the DB-drop flake
mitigation)")
462         p.add_argument("--update-baseline", action="store_true",
463                        help="freeze the current reel counts + quality as the new baseline
instead of checking")
464         p.add_argument("--email", action="store_true", help="email the report (see
scripts/nightly_email.py)")
465         p.add_argument("--email-to", default=None, help="override the report recipient")
466         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
467         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
468         return p
469
470
471     def main(argv: Optional[List[str]] = None) -> int:
472         import logging
473         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
474         log = logging.getLogger("nightly_reelcount")
475         args = build_parser().parse_args(argv)
476         tols = Tolerances(quality_tol=args.quality_tol)
477         date = args.date or _dt.date.today().isoformat()
478
479         if not os.path.exists(args.manifest):
480             log.error("manifest not found: %s", args.manifest)
481             return 2
482         manifest = load_manifest(args.manifest)
483         baseline = load_baseline(args.baseline)
484
485         try:
486             run_results, total_wall = run_corpus(manifest, baseline, rerun_on_mismatch=not
args.no_rerun_guard)
487         except Exception as e: # noqa: BLE001 ? operational failure (deps/config) ? exit 2,
never a false "pass"
488             log.error("run failed: %s", e)
489             return 2
490         if not run_results:

```



```

491         log.error("no videos in manifest %s", args.manifest)
492         return 2
493
494         billed = args.vm_uptime_seconds if args.vm_uptime_seconds is not None else
total_wall
495         cost = cost_report(
496             billed_seconds=billed,
497             machine_usd_per_hr=float(manifest.get("machine_usd_per_hr",
DEFAULT_MACHINE_USD_PER_HR)),
498             fx_inr_per_usd=float(manifest.get("fx_inr_per_usd", DEFAULT_FX_INR_PER_USD)),
499             gpu_seconds=total_wall,
500         )
501         segmenter = manifest.get("segmenter", "motion")
502         snapshot = summarize_results(run_results, segmenter, cost)
503         snapshot["git_commit"] = _git_head()
504         snapshot["generated_at"] = date
505
506         if args.update_baseline:
507             save_baseline(args.baseline, snapshot)
508             log.info("baseline frozen ? %s", args.baseline)
509             for r in run_results:
510                 log.info("  %s: reels=%s ok=%s", r["name"], r.get("reel_count"),
r.get("ok"))
511             return 0
512
513         if baseline is None:
514             log.error("no baseline at %s ? freeze one with --update-baseline (on the
configured engine).", args.baseline)
515             return 3
516
517         result = compare(baseline, snapshot, tols)
518         report = format_report(baseline, snapshot, result, date, snapshot["git_commit"])
519
520         if not args.no_report:
521             os.makedirs(args.report_dir, exist_ok=True)
522             md_path = os.path.join(args.report_dir, f"{date}.md")
523             with open(md_path, "w", encoding="utf-8") as fh:
524                 fh.write(report + "\n")
525             with open(os.path.join(args.report_dir, f"{date}.json"), "w", encoding="utf-8")
as fh:
526                 json.dump({"date": date, "status": result.overall_status(), "exit_code":
result.exit_code(),
527                           "snapshot": snapshot, "regressions": [f.message() for f in
result.regressions]},
528                           fh, indent=2, sort_keys=True)
529             log.info("report ? %s", md_path)
530
531         print(report)
532
533         if args.email:
534             try:
535                 from scripts.nightly_email import send_report
536                 subject = f"[KhelSutra nightly] {result.overall_status()} ? reel-count
engine regression {date}"
537                 sent = send_report(subject, report, to=args.email_to)
538                 log.info("email %s", "sent" if sent else "skipped (no creds ? report still
written)")
539             except Exception as e: # noqa: BLE001 ? a broken mailer must NOT mask the
regression verdict
540                 log.error("email failed (report still written): %s", e)
541
542         code = result.exit_code()
543         if code == 1:
544             log.error("REEL-COUNT REGRESSION (%d) ? see %s", len(result.regressions),
args.report_dir)

```

```

545         elif code == 4:
546             log.warning("baseline STALE ? corpus/segmenter changed; re-freeze with --update-
baseline")
547         else:
548             log.info("nightly reel-count check PASSED (cost $%.4f / ?%.2f)", cost["usd"],
cost["inr"])
549             return code
550
551
552     if __name__ == "__main__":
553         raise SystemExit(main())
554

```

11. assistant (2026-06-22T09:49:16.239Z)

Perfect! Now I have the critical evidence. Let me search for any other callers that might be affected:

12. user (2026-06-22T09:49:18.105Z)

```

Found 29 files
scripts\nightly_reelcount.py
tools\generate_highlights.py
tests\test_yolo_hybrid.py
tests\test_worker.py
tests\test_windowing.py
tests\test_trajectory_hybrid_equivalence.py
tests\test_tracknet_hybrid.py
tests\test_profile_parity.py
tests\test_output_base.py
tests\test_motion_segmenter.py
tests\test_job_worker_loop.py
tests\test_generate_highlights.py
tests\test_fusion_hybrid.py
tests\test_gemini.py
tests\test_base.py
tests\test_async_jobs.py
tests\test_api_endpoints.py
scripts\run_process_and_stitch.py
scripts\run_phase3_eval.py
backend\worker\__main__.py
backend\results.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\base.py
backend\main.py
backend\eval>windowing.py
backend\eval\served_gate_a.py

```

13. assistant (2026-06-22T09:49:20.232Z)

Let me now check how other segmenters declare their return types and examine the base class contract:

14. user (2026-06-22T09:49:21.639Z)

```

1     import os
2     import time
3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple, Callable

```

```

7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.utils.ffmpeg import ffmpeg_bin # P2a: single ffmpeg/ffprobe resolver
10    from backend.utils.paths import output_base
11    from backend.sports import sport_registry
12    from backend.providers import provider_registry
13    from backend.reporting import run_signals
14    from backend.results import Failure # Finding B: an unrunnable config (missing
key/unknown sport/provider) ? a loud Failure, not a silent 0-rally "success"
15
16    logger = logging.getLogger(__name__)
17
18
19    def _chunk_workdir(video_id: str, base: str = "output") -> str:
20        """Per-video chunk staging dir under <base>/chunks/<video_id> (`base` default
21        ``output``
22        keeps the prior literal byte-identical; the caller passes the configured output base
23
24        staging follows --output-dir ? COMPUTE_DECOUPLED_SERVING M3).
25
26        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
27        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
28        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
29        """
30        return os.path.join(base, "chunks", video_id)
31
32    def _parse_time(val) -> float:
33        """Converts a timestamp value to total float seconds.
34        Handles float/int pass-through, plain numeric strings, and
35        colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
36        when unexpected formats are encountered."""
37        if isinstance(val, (int, float)):
38            return float(val)
39        if isinstance(val, str):
40            val = val.strip()
41            if ":" in val:
42                # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal seconds.
43                # Convert it, but warn so the issue is visible in logs.
44                parts = val.split(":")
45                parts.reverse()
46                total = 0.0
47                for i, p in enumerate(parts):
48                    try:
49                        total += float(p) * (60 ** i)
50                    except ValueError:
51                        pass

```

15. assistant (2026-06-22T09:49:23.576Z)

Let me get the process_video method signature from gemini.py:

16. user (2026-06-22T09:49:24.381Z)

```

509: def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:

```

17. assistant (2026-06-22T09:49:29.848Z)

Interesting! Let me check the base class and see what the contract should be:

18. user (2026-06-22T09:49:30.744Z)

```

1     from abc import ABC, abstractmethod

```

```

2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():
55                 try:
56                     temp_inst = cls(None, {})
57                     desc = temp_inst.description
58                 except Exception:
59                     desc = "No description available."
60                 res[name] = desc
61             return res
62
63     segmenter_registry = SegmenterRegistry()

```

19. assistant (2026-06-22T09:49:34.164Z)

Excellent! Now I have the full picture. Let me check the actual behavior of how the nightly runner handles this (lines 345-350 in nightly_reelcount.py):

20. user (2026-06-22T09:49:35.125Z)

```

344         t0 = time.monotonic()
345         result = seg_cls(db, config).process_video(video_id, local)
346         wall = time.monotonic() - t0
347         if isinstance(result, Failure):
348             return None, None, wall, getattr(result, "message", "segmenter failed")

```

21. assistant (2026-06-22T09:49:37.159Z)

Let me also check how this is used in other places like main.py:

22. user (2026-06-22T09:49:38.103Z)

```

        segmenter = segmenter_cls(db_instance, global_config)
        try:
            result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
        except Exception:
            # A raising segmenter must not leave half-written rows: restore the pre-run
            # state (same destroy-after-confirm contract as the Failure branch), then let
            # the job worker record the failure.
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
--

@app.post("/api/process", status_code=202)
def process_video(req: ProcessRequest, request: Request):
    """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

    Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
    client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
    runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
--

    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
    segmenter = segmenter_cls(db_instance, global_config)
    result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
    segmentation_ran = True

    if isinstance(result, Failure):
        print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
        print("[FAIL] Indexing halted for video.")

```

23. assistant (2026-06-22T09:49:40.340Z)

Let me see the full context around the main.py usage:

24. user (2026-06-22T09:49:41.053Z)

```

522:         if isinstance(result, Failure):
1054:             if isinstance(result, Failure):

```

25. assistant (2026-06-22T09:49:42.978Z)

Let me check if there are any test failures or other direct callers:

26. user (2026-06-22T09:49:43.752Z)

```

1      """Characterization tests for MotionPlaySegmenter.process_video.
2
3      The motion segmenter is the legacy demo baseline: its detection half runs real
4      cv2 pixel-motion heuristics, and its DB-populate half *fabricates* segment
5      metadata/events with the stdlib ``random`` module. It carries ZERO direct tests
6      and is omitted from coverage (see .coveragerc).
7
8      These tests PIN today's behaviour so the detection/DB-populate halves can be
9      extracted into private helpers without changing observable output:
10
11     * ``cv2`` is mocked at the module level (``backend.pipeline.segmenters.motion.cv2``)
12       the way ``test_yolo_hybrid`` mocks ``cv2.VideoCapture`` ? but here the whole
13       module is faked because process_video calls cvtColor/GaussianBlur/absdiff/
14       threshold/dilate/countNonZero directly. The fake ``countNonZero`` drives a
15       scripted ``motion_ratio`` per analysed frame so the detected segment boundaries
16       are deterministic.
17     * The stdlib ``random`` is SEEDED (``random.seed(0)``) before each call so the
18       fabricated server/receiver/shot_count/events are reproducible. The random call
19       ORDER inside process_video is what the seeded assertions pin.
20     """
21     from unittest.mock import MagicMock, patch
22
23     import random
24
25     import backend.pipeline.segmenters.motion as motion_mod
26     from backend.pipeline.segmenters.motion import MotionPlaySegmenter
27     from backend.results import Success, Failure
28
29
30     # cv2 property constants the segmenter reads off the capture. The real values
31     # do not matter to the mock as long as the same constant maps to the same stub.
32     _CAP_PROP_FPS = "CAP_PROP_FPS"
33     _CAP_PROP_FRAME_COUNT = "CAP_PROP_FRAME_COUNT"
34
35
36     def _make_cv2_mock(motion_ratios, fps=30.0):
37         """Build a fake ``cv2`` module for the motion segmenter.
38
39         ``motion_ratios`` is the sequence of motion ratios the segmenter should
40         observe for the frames where ``prev_gray`` is set (i.e. every analysed frame
41         after the first). The capture yields ``len(motion_ratios) + 1`` readable
42         frames so that exactly ``len(motion_ratios)`` diffs are produced.
43
44         The fake ``countNonZero`` returns ``ratio * AREA`` and ``threshold`` returns a
45         stub whose ``.shape`` multiplies to ``AREA``, so
46         ``countNonZero / (shape[0] * shape[1]) == ratio``.
47         """
48         AREA_H, AREA_W = 100, 100 # area = 10_000
49         area = AREA_H * AREA_W
50
51         cv2_mock = MagicMock()
52         cv2_mock.CAP_PROP_FPS = _CAP_PROP_FPS
53         cv2_mock.CAP_PROP_FRAME_COUNT = _CAP_PROP_FRAME_COUNT
54         cv2_mock.COLOR_BGR2GRAY = "COLOR_BGR2GRAY"
55         cv2_mock.THRESH_BINARY = "THRESH_BINARY"
56
57         num_readable = len(motion_ratios) + 1
58
59         cap_mock = MagicMock()
60         cap_mock.isOpened.return_value = True
61
62         def mock_get(prop):
63             if prop == _CAP_PROP_FPS:
64                 return fps
65             if prop == _CAP_PROP_FRAME_COUNT:

```

```

66         # A large frame count so the frame_idx >= total_frames guard never
67         # short-circuits the read loop (reads end via the (False, None) tail).
68         return 1_000_000
69     return 0.0
70
71     cap_mock.get.side_effect = mock_get
72     cap_mock.grab.return_value = True # intermediate frames always grab-able
73     cap_mock.read.side_effect = (
74         [(True, "frame")] * num_readable + [(False, None)]
75     )
76
77     cv2_mock.VideoCapture.return_value = cap_mock
78
79     # A "gray" stand-in object carrying a .shape for the area computation.
80     thresh_stub = MagicMock()

```

27. assistant (2026-06-22T09:49:45.969Z)

Let me continue reading the test:

28. user (2026-06-22T09:49:49.547Z)

```

80     thresh_stub = MagicMock()
81     thresh_stub.shape = (AREA_H, AREA_W)
82
83     cv2_mock.cvtColor.return_value = "gray"
84     cv2_mock.GaussianBlur.return_value = "blurred"
85     cv2_mock.absdiff.return_value = "diff"
86     cv2_mock.threshold.return_value = (0.0, thresh_stub)
87     cv2_mock.dilate.return_value = thresh_stub
88
89     # countNonZero is called once per analysed frame that has a prev_gray, in
90     # order, so it walks the scripted motion_ratios.
91     counts = [int(round(r * area)) for r in motion_ratios]
92     cv2_mock.countNonZero.side_effect = counts
93
94     return cv2_mock, cap_mock
95
96
97 def _make_segmenter(db_mock=None, config=None):
98     if db_mock is None:
99         db_mock = MagicMock()
100     if config is None:
101         config = {"indexing": {}}
102     return MotionPlaySegmenter(db_mock, config)
103
104
105 def test_process_video_failure_on_unopenable_video():
106     """An un-openable capture returns a non-recoverable Failure, no DB writes."""
107     cv2_mock, cap_mock = _make_cv2_mock([0.0])
108     cap_mock.isOpened.return_value = False
109
110     db_mock = MagicMock()
111     segmenter = _make_segmenter(db_mock=db_mock)
112
113     with patch.object(motion_mod, "cv2", cv2_mock):
114         result = segmenter.process_video("vid-1", "bad.mp4")
115
116     assert isinstance(result, Failure)
117     assert result.message == "Could not open video: bad.mp4"
118     assert result.is_recoverable is False
119     db_mock.add_play_segment.assert_not_called()
120     db_mock.add_event.assert_not_called()
121
122

```

```

123 def test_process_video_no_motion_returns_empty_success():
124     """All-quiet motion (below threshold) yields a Success with no segments."""
125     # 30 analysed frames all well below the 0.08 motion_threshold.
126     cv2_mock, _ = _make_cv2_mock([0.0] * 30)
127     db_mock = MagicMock()
128     segmenter = _make_segmenter(db_mock=db_mock)
129
130     with patch.object(motion_mod, "cv2", cv2_mock):
131         result = segmenter.process_video("vid-1", "quiet.mp4")
132
133     assert isinstance(result, Success)
134     assert result.value == []
135     db_mock.add_play_segment.assert_not_called()
136
137
138 def _high_motion_ratios():
139     """A motion profile that produces exactly one segment long enough to survive
140     the 3.0s min_segment_duration filter.
141
142     fps=30, analysis_fps=5 -> frame_interval=6, so each analysed frame is 6 real
143     frames apart -> 0.2s of video per analysed frame. We drive ~40 high-motion
144     analysed frames (~8s) so the smoothed signal stays above threshold for a
145     span well over 3s.
146     """
147     return [0.5] * 40
148
149
150 def test_process_video_fabricates_one_segment_deterministically():
151     """With seeded ``random`` the fabricated segment + metadata are pinned exactly."""
152     cv2_mock, _ = _make_cv2_mock(_high_motion_ratios())
153
154     db_mock = MagicMock()
155     # add_play_segment returns a deterministic id so the returned dict is pinned.
156     db_mock.add_play_segment.return_value = 42
157
158     segmenter = _make_segmenter(db_mock=db_mock)
159
160     with patch.object(motion_mod, "cv2", cv2_mock):
161         random.seed(0)
162         result = segmenter.process_video("vid-1", "active.mp4")
163
164     assert isinstance(result, Success)
165     segments = result.value
166
167     # === PIN: exactly one fabricated segment survives the min-duration filter ===
168     assert len(segments) == 1
169     seg = segments[0]
170
171     # Pin the full returned dict shape/values (boundaries come from the motion
172     # loop, server/receiver/metadata from seeded random).
173     assert seg == _EXPECTED_SEGMENT
174
175     # === PIN: the DB side effects ===
176     db_mock.add_play_segment.assert_called_once()
177     ps_args, ps_kwargs = db_mock.add_play_segment.call_args
178     assert _EXPECTED_ADD_PLAY_SEGMENT_ARGS == (ps_args, ps_kwargs)
179

```

29. assistant (2026-06-22T09:49:52.343Z)

Perfect! Now I have all the evidence I need. The type hint defect is confirmed. Let me check one more thing - if there are any other segmenters to see if they have the same issue:

30. user (2026-06-22T09:49:54.242Z)


```
def process_video(self, video_id: str, video_path: str, # type: ignore[override]
                  player_pool: Optional[List[str]] = None,
                  max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
    # override-ignore: the ABC declares the Result contract (Success|Failure)
```

31. assistant (2026-06-22T09:49:56.402Z)

Ah! This is interesting - trajectory_hybrid explicitly has a `# type: ignore[override]` comment!
Let me check yolo_hybrid:

32. user (2026-06-22T09:49:57.313Z)

```
def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
    self.person.lazy_load_model()

    # Phase 0: resolve sport profile + AI provider (early-returns [] on any failure).
```

33. assistant (2026-06-22T09:49:59.084Z)

Let me check the full trajectories for yolo_hybrid and gemini:

34. user (2026-06-22T09:49:59.934Z)

```
def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
    """
    Processes a video to detect play segments using Google Gemini Multimodal Video API,
```

35. assistant (2026-06-22T09:50:01.914Z)

Now let me verify the actual runtime behavior by searching for what these implementations return:

36. user (2026-06-22T09:50:03.030Z)

```
86:         return Failure(str(e))
102:         return Failure(msg)
108:         return Failure(str(e))
```

37. assistant (2026-06-22T09:50:04.907Z)

Let me check yolo_hybrid:

38. user (2026-06-22T09:50:05.977Z)

(Bash completed with no output)

39. assistant (2026-06-22T09:50:07.912Z)

Let me check what yolo_hybrid actually returns:

40. user (2026-06-22T09:50:08.634Z)

```
1      """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2      (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4      Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5      (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6      can be consumed by the future fusion segmenter; this segmenter just orchestrates
7      chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9      Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
```

```

10     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11     detection uses torchvision's permissively-licensed (BSD) detectors and the
12     association/tracking that YOLO.track() used to bundle in is provided by
13     supervision's MIT-licensed ByteTrack.
14
15     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16     DB `detector` tags, even though no YOLO is involved anymore.
17     """
18     import os
19     import time
20     import logging
21     import concurrent.futures
22     from typing import List, Dict, Any, Tuple
23
24     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25     from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26     from backend.utils.video import get_video_duration, extract_video_chunk
27     from backend.utils.paths import output_base
28     from backend.sports import sport_registry
29     from backend.providers import provider_registry
30
31     logger = logging.getLogger(__name__)
32
33
34     def _fmt_ts(seconds: float) -> str:
35         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
36         seconds = max(0, int(seconds))
37         h, rem = divmod(seconds, 3600)
38         m, s = divmod(rem, 60)
39         return f"{h:02d}:{m:02d}:{s:02d}"
40
41
42     class HybridYoloSegmenter(BasePlaySegmenter):
43         def __init__(self, db, config):
44             super().__init__(db, config)
45             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
46             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
47             self.person = PersonDetector(config)
48
49             @property
50             def name(self) -> str:
51                 return "yolo_hybrid"
52
53             @property
54             def description(self) -> str:
55                 return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
56                     "candidate filtering, then an AI provider for high-precision semantic
57                     boundaries.")
58
59             def _resolve_provider(self, video_path: str):
60                 """Phase 0: resolve sport profile + AI provider (with API key).
61
62                 Returns the configured provider instance, or ``None`` to signal that
63                 ``process_video`` should early-return ``[]`` (bad sport, missing API key,
64                 or unknown provider). Behaviour is identical to the inlined version.
65                 """
66                 # Get sport profile config
67                 sport_name = self.config.get("sport", "badminton")
68                 try:
69                     sport_registry.get(sport_name)
70                 except KeyError as e:
71                     logger.error(str(e))
72                     return None

```

```

73         # Get AI provider and model config
74         idx_cfg = self.config.get("indexing", {})
75         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
76         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
77
78         # Get appropriate API key based on the provider
79         api_key_env_var = f"{provider_name.upper()}_API_KEY"
80         api_key = os.environ.get(api_key_env_var)
81         if not api_key:
82             from backend.pipeline.segmenters._keyhint import api_key_hint
83             logger.error(
84                 f"{api_key_env_var} environment variable is not set! "
85                 f"To run the {provider_name} segmenter, set your API key. "
86                 + api_key_hint(api_key_env_var)
87             )
88             return None
89
90         try:
91             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
92         except KeyError as e:
93             logger.error(str(e))
94             return None
95
96         logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
97         return provider
98
99     def _collect_candidate_windows(
100         self, video_id: str, video_path: str, video_duration: float, idx_cfg: Dict[str,
Any],

```

41. assistant (2026-06-22T09:50:10.797Z)

Let me search for the return statements in yolo_hybrid:

42. user (2026-06-22T09:50:12.346Z)

```

39:     return f"{h:02d}:{m:02d}:{s:02d}"
51:     return "yolo_hybrid"
55:     return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast ")
71:     return None
88:     return None
94:     return None
97:     return provider
135:         return (ci, cs, chunk_path, success)
195:         return {
220:     return all_candidate_windows
258:     return candidate_ids
283:         return []
305:         return valid_segments
319:     return gemini_segments
364:     return segments_data
372:         return []
383:         return []
435:     return self._populate_db(video_id, gemini_segments, model_name)

```

43. assistant (2026-06-22T09:50:15.247Z)

Let me check the end of process_video in yolo_hybrid:

44. user (2026-06-22T09:50:15.906Z)

```

400         "merge_gap": merge_gap,
401         "frame_skip": frame_skip,
402         "tracker": tracker_config,
403         "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
404         "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
405         "min_action_window_duration": idx_cfg.get("min_action_window_duration",
406 2.0),
407     }
408     chunks_dir = os.path.join(output_base(self.config), "chunks_yolo")
409     os.makedirs(chunks_dir, exist_ok=True)
410
411     # Phase 2: chunk ? per-chunk action windows ? cross-chunk overlap merge.
412     all_candidate_windows = self._collect_candidate_windows(
413         video_id, video_path, video_duration, idx_cfg,
414         chunk_duration, chunk_overlap, velocity_thresh, merge_gap,
415         frame_skip, chunks_dir,
416     )
417
418     # Phase 2b: log + persist raw candidates (window index -> candidate_id).
419     candidate_ids = self._persist_candidates(
420         video_id, all_candidate_windows, detection_params, log_candidates,
421         store_candidates,
422     )
423
424     try:
425         os.rmdir(chunks_dir)
426     except Exception:
427         pass
428
429     # Phase 3: AI Provider Handoff
430     gemini_segments = self._run_ai_handoff(
431         video_id, video_path, video_duration, all_candidate_windows, candidate_ids,
432         provider, sport_profile, provider_name, chunk_duration, handoff_padding,
433         ai_max_workers,
434     )
435
436     # Phase 4: DB Population
437     return self._populate_db(video_id, gemini_segments, model_name)
438
439     # Register the segmenter
440     segmenter_registry.register(HybridYoloSegmenter)

```

45. assistant (2026-06-22T09:50:18.709Z)

Now let me check the `_populate_db` method to see what it returns:

46. user (2026-06-22T09:50:19.562Z)

```

340         metadata = {
341             "shot_count": shot_count,
342             "shot_count_confidence": segment.get("shot_count_confidence",
343 "Unknown"),
344             "detector": f"yolo-hybrid-{model_name}"
345         }
346         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
347 metadata)
348         if segment_id:
349             # Link back to the candidate that produced this segment. A candidate
350             # may yield multiple AI segments (one-to-many); confirmed_segment_id
351             keeps
352             # the last one ? the link table records "this candidate was
353             confirmed".

```

```

351         candidate_id = segment.get("_candidate_id")
352         if candidate_id is not None:
353             self.db.link_candidate_to_segment(candidate_id, segment_id)
354         segments_data.append({
355             "id": segment_id,
356             "start_time": start,
357             "end_time": end,
358             "duration": dur,
359             "metadata": metadata
360         })
361     except Exception as e:
362         logger.warning(f"Skipping invalid segment struct: {e}")
363
364     return segments_data
365
366     def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
367         self.person.lazy_load_model()
368
369         # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
370         provider = self._resolve_provider(video_path)
371         if provider is None:
372             return []
373
374         # Re-read the config values needed downstream (pure deterministic reads).
375         idx_cfg = self.config.get("indexing", {})
376         sport_profile = sport_registry.get(self.config.get("sport", "badminton"))
377         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
378         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
379
380         video_duration = get_video_duration(video_path)
381         if video_duration <= 0:
382             logger.error("Invalid video duration.")
383             return []
384
385         chunk_duration = idx_cfg.get("chunk_duration", 300)
386         chunk_overlap = idx_cfg.get("chunk_overlap", 30)
387         velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
388         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
389         frame_skip = idx_cfg.get("yolo_frame_skip", 3)

```

47. assistant (2026-06-22T09:50:24.817Z)

Perfect! Now I have the complete picture. The `yolo\_hybrid.process\_video` method:

- Line 366 declares return type as `List[Dict[str, Any]]`
- Line 372 returns `[]` (an empty list) on error
- Line 383 returns `[]` (an empty list) on error
- Line 435 returns the result of `\_populate\_db()` which returns `segments\_data` (line 364), a list

So `yolo\_hybrid` does NOT return Success/Failure - it violates the base class contract by returning bare lists.

Now let me verify the key issue: the type hint in motion.py is wrong. Let me create a comprehensive summary of my findings:

48. user (2026-06-22T09:50:38.304Z)

Structured output provided successfully

Summary

****REAL DEFECT CONFIRMED**** ?

The finding is accurate. Motion.py has a critical type hint violation:

****The Problem:****

- ****Line 20**** declares return type as `List[Dict[str, Any]]``
- ****Actual implementation**** returns `Success | Failure`` (lines 31 and 37)
- ****Base class contract**** (base.py:24) requires `Result[List[Dict[str, Any]]]`` which is `Success[List[Dict[str, Any]]] | Failure``

****Evidence:****

1. Tests (test_motion_segmenter.py) explicitly verify `isinstance(result, Success)`` and `isinstance(result, Failure)`` ? they expect the Success/Failure union
2. Nightly runner (nightly_reelcount.py:347-349) handles this correctly with `isinstance(result, Failure)`` checks
3. Base class (base.py:24) mandates the Result type hint
4. Other segmenters have acknowledged this issue: trajectory_hybrid uses `# type: ignore[override]``, but motion.py doesn't

****Runtime Impact:**** While the nightly runner handles this correctly via defensive isinstance checks, any code that treats the result as a bare list (e.g., `len(result)``) would crash with `TypeError` when receiving a Success dataclass object instead of a list.

****Severity: HIGH**** ? Type contract violation that escapes to callers and violates the Liskov substitution principle defined in the base class.